



Machine Learning

HOMEWORK 2

NAME:

STUDENT ID#:

Objectives:

- Understand and implement the forward pass of a **Deep Neural Network (DNN)** using matrix operations.
- Apply **activation functions**, **softmax**, and **cross-entropy loss** for **classification tasks**.
- Understand and implement **Gradient Descent (GD)**-based optimization methods, including Mini-batch **SGD**, **Momentum**, and **Adam**.
- Understand the concept of **Backpropagation** and how gradients are computed and propagated through a neural network.
- Analyze the effect of different network structures and optimization methods on model performance.
- Evaluate classification results using standard metrics such as **accuracy**, **confusion matrix**, **precision**, **recall**, **F1-score**, and **ROC**.

Part 1. Instruction

1. Deep Neural Network.

In this assignment, you will build a **multilayer feedforward neural network** using **only NumPy and Matplotlib** to solve a **multi-class classification** problem on the **MNIST dataset**. You must use the **dataset provided** by the TA. Implement the forward pass only, **without backpropagation**. Evaluate your model using **accuracy**, **confusion matrix**, and **ROC curves**. You need to try experiments with different activation functions and network structures to improve your model's performance.

2. Gradient Descent.

In this assignment, you will implement **Mini-batch Stochastic Gradient Descent (SGD)** and its extensions using Algorithms 7.2, 7.3, and 7.4. Your task is to build a binary classifier to determine whether an **MNIST image** matches a specific digit or not (e.g., "Is this a 4 or not?"). You will implement three different methods: **Mini-batch SGD (Algorithm 7.2)**, **SGD with Momentum (Algorithm 7.3)** and **Adam Optimizer (Algorithm 7.4)**. The classifier must use the **sigmoid activation function** and be implemented entirely with NumPy. Do not use external machine learning libraries (e.g., scikit-learn, PyTorch). The model should output: Final accuracy on the test set, and five misclassified test samples, with true and predicted labels shown. Use the last digit of your student ID as the **TARGET_DIGIT** for binary classification (e.g., ID ending in 7 $\rightarrow$ **TARGET_DIGIT** = 7).

3. Backpropagation.

In this assignment, please **train a logistic classifier** to recognize whether an MNIST digit image is the target digit (e.g., "Is it a 3?") or not. You will integrate a backpropagation autodiff mechanism into the SGD training loop to compute gradients used for weight updates.

Integrate an **autodiff module** that traces:



- **Primal values** through the forward pass (e.g., intermediate variables like $v_3 = x_1 x_2$).
- **Forward-mode** using the chain rule from inputs to output.
- **Reverse-mode** representing the backpropagation path from output to inputs.

Use only NumPy for all computations. Do not use libraries like scikit-learn or PyTorch. The dataset and sample code can be found here: <https://github.com/Satriosnjya/ML-Labs.git>

Book References: Deep Learning: Foundations and Concepts 2024th Edition

Part 3. Arithmetic Instructions

A. Deep Neural Network

Step **Procedure**

1 Activation Function- Refer to equations 6.14 – 6.18

2 Multilayer Feedforward
 1. Refer to Equation 6.7-6.9
 2. Equation 6.19:

$$z^{(l)} = h^{(l)}(W^{(l)} z^{(l-1)})$$

3 Cross Entropy and one-hot encoding – Equation 6.36:

$$E(w) = - \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w)$$

4 SoftMax Function – Equation 6.37:

$$y_k(x, w) = \frac{\exp(a_k(x, w))}{\sum_j \exp(a_j(x, w))}$$

5 Evaluation using Confusion Matrix, ROC, Accuracy, Precision, Recall. Refer to the previous Homeworks

B. Gradient Descent

Procedure

Algorithm 7.2: Mini-batch stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$

Batch size B

Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$

Learning rate parameter η

Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$

repeat

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$ // weight vector update

$n \leftarrow n + B$

if $n > N$ **then**

 shuffle data

$n \leftarrow 1$

end if

until convergence

return $\mathbf{w}$



Algorithm 7.3: Stochastic gradient descent with momentum

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
 Batch size B
 Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
 Learning rate parameter η
 Momentum parameter μ
 Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

```

 $n \leftarrow 1$ 
 $\Delta \mathbf{w} \leftarrow \mathbf{0}$ 
repeat
   $\Delta \mathbf{w} \leftarrow -\eta \nabla E_{n:n+B-1}(\mathbf{w}) + \mu \Delta \mathbf{w}$  // calculate update term
   $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$  // weight vector update
   $n \leftarrow n + B$ 
  if  $n > N$  then
    shuffle data
     $n \leftarrow 1$ 
  end if
until convergence
return  $\mathbf{w}$ 

```

Algorithm 7.4: Adam optimization

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
 Batch size B
 Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
 Learning rate parameter η
 Decay parameters β_1 and β_2
 Stabilization parameter δ

Output: Final weight vector $\mathbf{w}$

```

 $n \leftarrow 1$ 
 $\mathbf{s} \leftarrow \mathbf{0}$ 
 $\mathbf{r} \leftarrow \mathbf{0}$ 
repeat
  Choose a mini-batch at random from  $\mathcal{D}$ 
   $\mathbf{g} = -\nabla E_{n:n+B-1}(\mathbf{w})$  // evaluate gradient vector
   $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1) \mathbf{g}$ 
   $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2) \mathbf{g} \odot \mathbf{g}$  // element-wise multiply
   $\hat{\mathbf{s}} \leftarrow \mathbf{s} / (1 - \beta_1^t)$  // bias correction
   $\hat{\mathbf{r}} \leftarrow \mathbf{r} / (1 - \beta_2^t)$  // bias correction
   $\Delta \mathbf{w} \leftarrow -\eta \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$  // element-wise operations
   $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$  // weight vector update
   $n \leftarrow n + B$ 
  if  $n + B > N$  then
    shuffle data
     $n \leftarrow 1$ 
  end if
until convergence
return  $\mathbf{w}$ 

```



Part 4. Data Transfer Instructions

Procedure

- 1 Download the dataset and code from: <https://github.com/Satriosnjya/ML-Labs.git>
- 2 Open colab.research.google.com or you may use your **local Python environment**.
- 3 Create a new notebook by clicking “New Notebook.”
- 4 For the **DNN** task: The example code can be found in the “Homework 2/1. DNN” folder.
- 5 For the **GD** task: The example code can be found in the “Homework 2/2. GD” folder.
- 6 For the **Backpropagation** task: The example code can be found in the “Homework 2/ 3. Backpropagation ” folder.

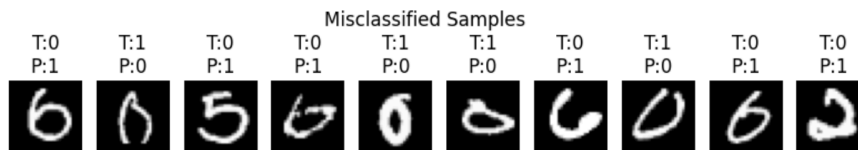
Example Output (Just for reference):

Gradient Descent

```
[INFO] Header: 60000 images, 28x28
[INFO] Loading 60000 images based on file size
[INFO] Loading 60000 labels based on file size
[INFO] Header: 10000 images, 28x28
[INFO] Loading 10000 images based on file size
[INFO] Loading 10000 labels based on file size
```

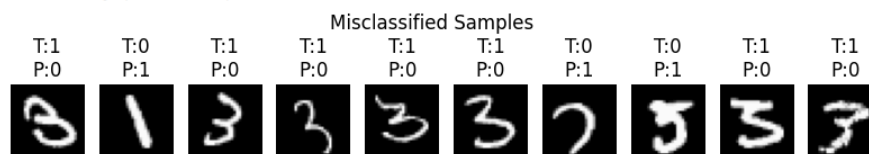
[INFO] Binary classification: '0' vs not-0

Test Accuracy (is 0 or not): 0.9927



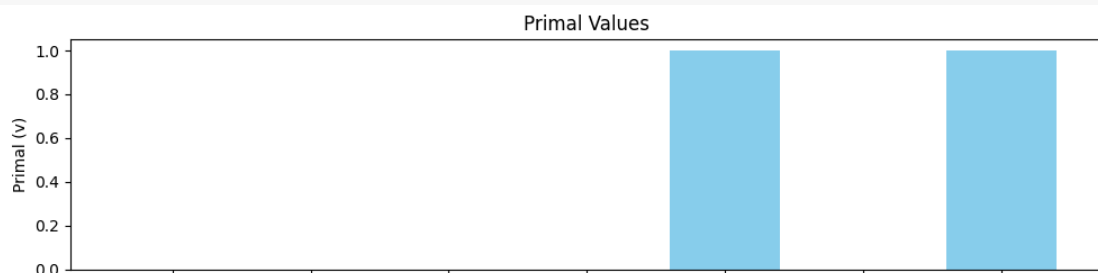
Backpropagation

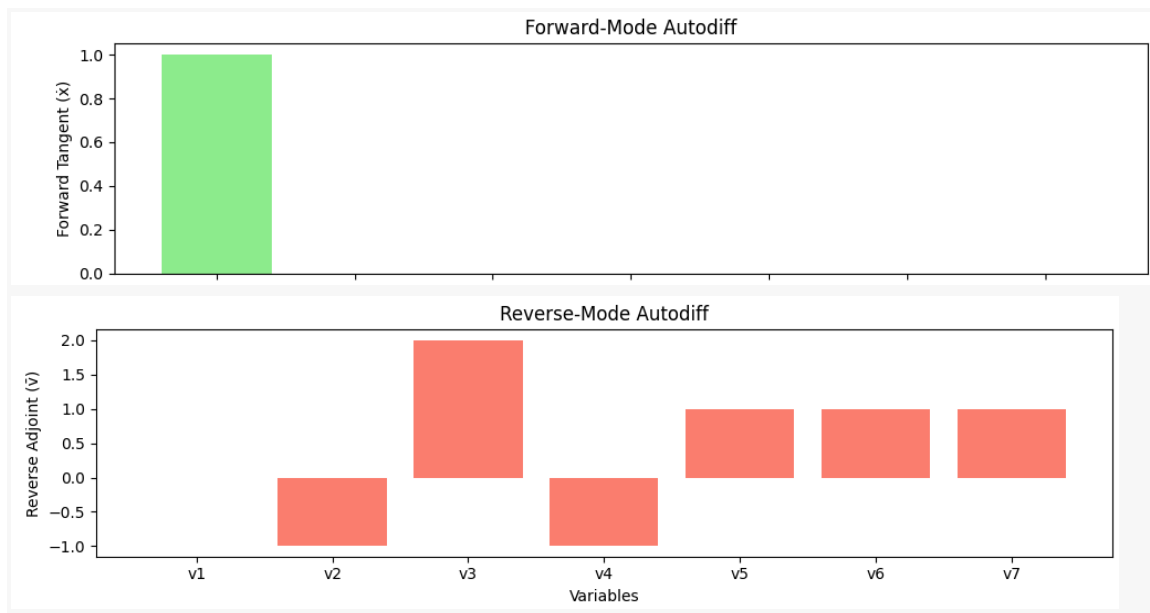
Test Accuracy (is 3 or not): 0.9774



Autodiff Trace Table (sample features):

Variable	Primal (v)	Forward Tangent (ẋ)	Reverse Adjoint (ẏ)
0 v1	0.0	1.0	0.0
1 v2	0.0	0.0	-1.0
2 v3	0.0	0.0	2.0
3 v4	0.0	0.0	-1.0
4 v5	1.0	0.0	1.0
5 v6	0.0	0.0	1.0
6 v7	1.0	0.0	1.0





Grading & Submission Instructions

Your report must present **results, figures, and analysis** based on the implementation tasks. The report should follow the outline below.

1. (40%) Deep Neural Network

- 1.1. (7.5%) Implement a Feedforward Neural Network with More Than One Hidden Layer.
- 1.2. (7.5%) Activation functions implemented from scratch (Eq. 6.14–6.18), as well as Softmax output and cross-entropy loss (Eq. 6.36 & 6.37).
- 1.3. (7.5%) Model runs correctly and generates prediction results.
- 1.4. Evaluation:
 - 1.4.1. (2.5%) Confusion matrix (plotted),
 - 1.4.2. (2.5%) ROC curve for 10 classes (plotted),
 - 1.4.3. (2.5%) Precision, Recall, F1, Overall Accuracy
- 1.5. **Question:**
 - 1.5.1. (5%) Explain how you designed your model (number of layers, neurons, and activation functions). What changes did you make to improve the accuracy, and how did those changes affect the results? ***Please attach the performance results before and after your improvements.**
 - 1.5.2. (5%) Based on your evaluation results (confusion matrix, ROC, etc.), how well did the model perform? Which classes are harder to predict? Why do you think that happened?

2. (30%) Gradient Descent

- 2.1. Correctly implemented, runs, shows accuracy and sample misclassification of:
 - 2.1.1. (5%) Mini-batch SGD (Algorithm 7.2)
 - 2.1.2. (5%) SGD with momentum (Algorithm 7.3)
 - 2.1.3. (5%) Adam Optimizer (Algorithm 7.4)
- 2.2. (7.5%) Compare the accuracy and misclassified test samples for each algorithm.
- 2.3. **Question:**
 - 2.3.1. (2.5%) Which optimizer gave you the best test accuracy? Why do you think it performed better than the others?



2.3.2. **(2.5%)** What is the differences in learning stability, convergence speed, or misclassification types across the three algorithm? Please explain with examples or observation from your results.

2.3.3. **(2.5%)** How did your choice of learning rate, batch size, or momentum affect each optimizer? What values worked best in your experiments?

3. (30%) Backpropagation

3.1. **(10%)** Implement the backpropagation autodiff

3.2. **(10%)** The model runs successfully without errors, use the provided MNIST dataset, and output the primal values, forward and reverse mode autodiff

3.3. **(5%)** Set the class of binary classification to the last digit of your student ID. (e.g., if your ID ends in 5, use the class '5'). Displays the result as shown as the "Example Output" in the last pages of this document.

3.4. **(5%)** Briefly discuss your results.

Submission:

1. **Report:** The **report must be written in English**. Include screenshots of your results for each task (model training, evaluation, and plots).
2. **Code:** Submit your complete Python script (.py or .ipynb notebook).
3. Upload both your report and code to the E3 system. Name your files correctly as StudentID_HW2.zip with the following directory structure:

```
StudentID_HW2.zip |
├── 1. DNN
│   ├── Report: StudentID_report.pdf
│   ├── Code: StudentID_code.py or StudentID_code.ipynb
│   └── Dataset.zip
├── 2. GD
│   ├── Report: StudentID_report.pdf
│   ├── Code: StudentID_code.py or StudentID_code.ipynb
│   └── Dataset.zip
└── 3. Backpropagation
    ├── Report: StudentID_report.pdf
    ├── Code: StudentID_code.py or StudentID_code.ipynb
    └── Dataset.zip
```

4. Submission Rules:
 - 1 day late: **10% deduction** from the total score.
 - Incorrect file name: **10% deduction**.
 - Incorrect directory structure: **10% deduction**.
5. Plagiarism is **strictly prohibited**. Copying reports or code from **other students, previous submissions, or any external sources will result in penalties**.

